



Birzeit University

Faculty of Engineering and Technology
Department of Electrical and Computer Engineering

Applied Cryptography (ENCS4320)

SecureChannel

A Secure Communication System

Omar Takhman

1230002

1 Project Summary

The program is a secure messaging program written in Python. Two parties (a client and a server) connect over a TCP socket, run a handshake to agree on keys, and then exchange messages that are encrypted, authenticated, and protected against tampering and replay.

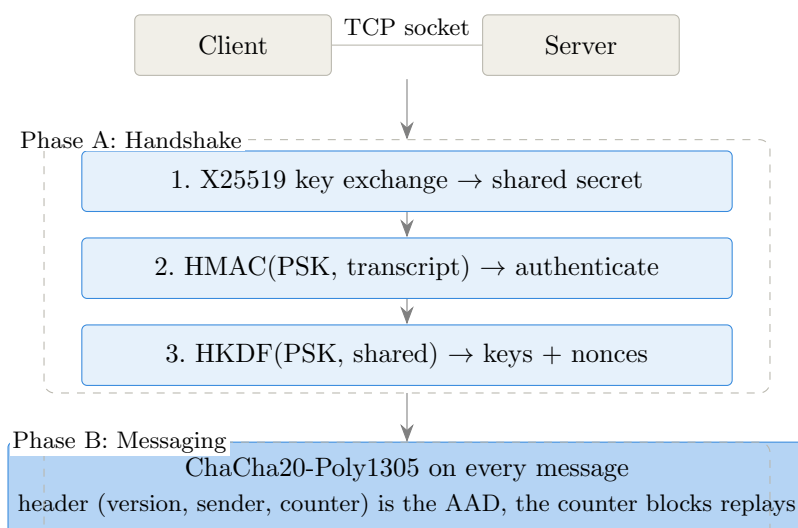
The point of the project is that every cryptographic building block is implemented by hand from its official specification, with no cryptographic libraries used inside the implementation. The same primitives that secure real systems such as TLS 1.3, SSH, Signal, and WireGuard are built here from the ground up and validated against the official test vectors published in their RFC and FIPS documents.

The code is organised in layers, each one building on the one below it:

- **primitives/** holds the individual building blocks, each from one standard: SHA-256, HMAC, HKDF, ChaCha20, Poly1305, the ChaCha20-Poly1305 AEAD, and X25519.
- **protocol/** ties the primitives together into a real conversation: the handshake, the key schedule, and the secure channel.
- **app/** contains `peer.py`, the command line chat a user runs.
- **tests/** has one test file per stage, each checked against the official standard's test vectors.

2 System Overview

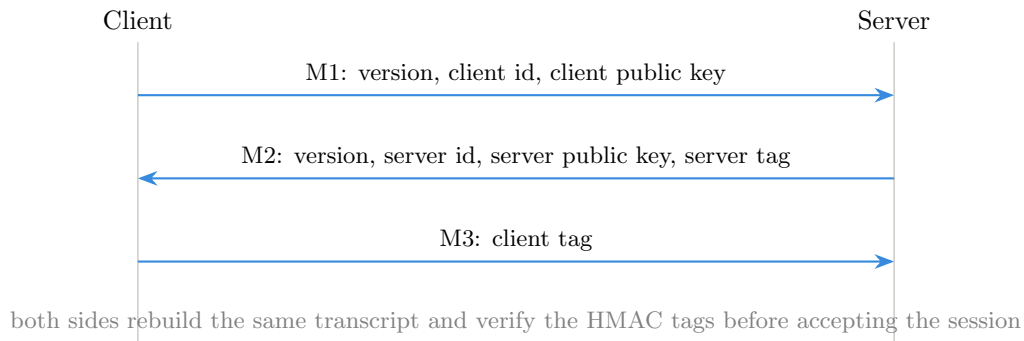
A session runs in two phases. In Phase A the two sides perform the handshake: an X25519 key exchange gives them a shared secret, an HMAC tag computed over the whole handshake transcript with the pre shared key (PSK) proves each side knows the password, and HKDF turns the shared secret into the session keys. In Phase B every application message is protected with ChaCha20-Poly1305, the plaintext header is authenticated as associated data, and a strictly increasing counter blocks replays.



3 Protocol Design

3.1 Handshake and message formats

The handshake is three messages. Every field is length prefixed on the wire: each field is written as a 2byte big endian length followed by the field bytes, and the whole message is framed with a 4byte big endian total length, so the receiver always knows exactly how many bytes to read.



Each side generates an ephemeral X25519 key pair. The transcript is the protocol version, both identities, and both public keys, concatenated with the same length prefixing. The server proves it knows the PSK by sending $\text{HMAC}(\text{PSK}, \text{transcript} + \text{"server"})$ and the client proves it with $\text{HMAC}(\text{PSK}, \text{transcript} + \text{"client"})$. If a tag does not match, the side aborts the connection. Because the ephemeral public keys are inside the transcript that the tags cover, an attacker in the middle cannot swap in their own keys without knowing the PSK, which is what defeats a man in the middle attack.

3.2 Key schedule

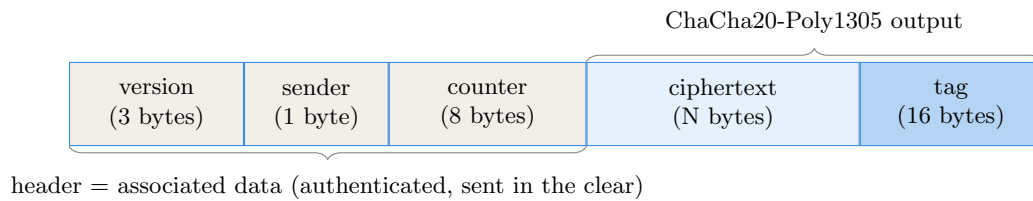
The X25519 shared secret is turned into the session keys with HKDF, and the PSK is mixed in as the HKDF salt, so both the key exchange and the shared password are needed to get the right keys. HKDF runs in two steps: extract compresses the inputs into one clean pseudorandom key (the PRK), and expand stretches the PRK into four separate outputs, each with a different label so they never collide:

- `client_to_server_key` (32 bytes)
- `server_to_client_key` (32 bytes)
- `client_to_server_iv` (12 bytes)
- `server_to_client_iv` (12 bytes)

Each direction has its own key and its own IV, so the two directions never share key material.

3.3 Secure channel and message format

After the handshake, every message is a 12byte plaintext header followed by the ChaCha20-Poly1305 output (the ciphertext with a 16byte tag appended), the header is the associated data it is sent in the clear but it is authenticated, so any change to it makes decryption fail.



3.4 Nonce construction

ChaCha20 needs a nonce that never repeats for a given key. The nonce is the direction's IV XORed with the message counter (the counter padded to 12 bytes). Because the counter only ever goes up during a session, the nonce is never reused:

```
counter_bytes = bytes(4) + counter.to_bytes(8, "big") # 4 zero bytes + 8byte counter
nonce = bytes(a ^ b for a, b in zip(base_iv, counter_bytes))
```

The same counter is also placed in the header, so the receiver reads it back and rebuilds the exact same nonce to decrypt.

3.5 Preventing replay and tampering

Tampering. Poly1305 produces a 16byte tag over the header and the ciphertext. On receive, the tag is recomputed and compared; if even one byte of the header or ciphertext changed, the tags differ and the message is rejected (decryption raises an error).

Replay and reordering. Each session remembers the highest counter it has accepted. A message whose counter is less than or equal to that value is dropped, so a captured message cannot be replayed and out of order messages are rejected:

```
if counter <= session["last_seen"]:
    raise ValueError("replay: counter %d was already seen" % counter)
```

Man in the middle. As described above, the PSK keyed HMAC over the full transcript binds the exchanged ephemeral keys to the two legitimate parties. A wrong PSK produces a wrong tag and the handshake is aborted.

4 Cryptographic Primitives

Every primitive below is implemented from scratch from its standard and checked against that standard's official test vectors.

Component	Specification	What it gives us
SHA-256	FIPS 180-4	secure hashing
HMAC	RFC 2104	message authentication
HKDF	RFC 5869	deriving keys from one shared secret
ChaCha20-Poly1305	RFC 8439	authenticated encryption (with AAD)
X25519	RFC 7748	Diffie-Hellman key exchange

5 Testing

The test suite validates each primitive against the official RFC and FIPS known answer vectors, which is how the project proves each one is implemented correctly. `test_phase1` covers SHA-256, HMAC, and HKDF `test_2` covers ChaCha20, Poly1305, and the AEAD; and `test_x25519` covers X25519. In addition, `test_protocol` runs a full handshake over a local socket pair and checks that messaging works in both directions, that a tampered message is rejected, that a replayed message is rejected, and that a wrong PSK is rejected. Every test prints `pass` when it succeeds.

6 AI Usage

AI was used in this project in the following ways:

- Getting an overview of how the different parts (primitives, protocol, app, tests) fit together.
- Outlining the steps and tasks needed to move the project forward (a what to do summary).
- Most of the test vectors used to validate the cryptographic implementations were provided by AI
- Explaining some code snippets found on the internet that I did not understand on my own.
- Generating the figures used in this report and in the documentation to illustrate the system and explain the code.
- Helping me commit and push to my own GitHub account with Claude Code (thats how I usually do it).

Most of the above was done using NotebookLM and Claude.

7 Contribution Statement

I designed and implemented the entire project. The cryptographic primitives, the protocol layer, the chat application, and the tests are all my own work, AI tools (NotebookLM and Claude) were to help understanding.